Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Yığın ağacı (heap) veri yapısının tanımı. Dönemin 7. haftası.
- Complete binary tree hatırlatması.
- Max heap ve min heap koşulları.
- Heap'in dizi ile gösterimi: çocuk ve ebeveyn indisleri.
- Heap'in yüksekliği ($\lg n$).
- En büyük elemanı okuma ($O(1)$).
- Heap oluşturma (build max heap): aşağıdan yukarı (bottom-up) yaklaşım ve `MAX-HEAPIFY`.
- En büyük elemanı heap'ten alma (extract max).
- Bir düğümün değerini artırma (increase key).
- Heap'e eleman ekleme (insert) ve yukarıdan aşağı (top-down) heap oluşturma.
- Yığın sıralaması (heap sort) ve "dönüştür ve fethet" (transform and conquer) yaklaşımı.
- Üçlü heap (ternary heap) ve genel d'li heap.

Hocanın Özellikle Vurguladığı Kısımlar

- Heap iki koşulu aynı anda sağlar:
  - Complete binary tree olmalıdır.
  - Max heap'te her ebeveyn çocuklarından büyük ya da eşittir. Min heap'te küçük ya da eşittir.
- Heap yalnız tek bir şeyi garanti eder: en büyük (max heap) ya da en küçük (min heap) eleman köktedir.
- Heap bir ikili arama ağacı (BST) değildir. Sol alt ağaçta küçükler, sağda büyükler diye bir kural yoktur. Kardeşler arasında sıra yoktur.
- Veri yapısı olan heap, Java gibi dillerdeki bellek alanı "heap space" ile aynı şey değildir.
- Türkçede "yığın/yığıt" hem stack hem heap için kullanılabildiği için derste İngilizce adlar tercih ediliyor.
- Ağacın yüksekliği $\lg n$ olduğu için heap üzerindeki ekleme, silme ve düzeltme işlemleri en kötü durumda $O(\log n)$'dir.
- Bottom-up ve top-down yöntemleriyle aynı diziden oluşturulan heap'ler aynı olmak zorunda değildir. İkisi de geçerli heap'tir.
- Heap sort, problemi başka bir gösterime dönüştürüp o gösterimin özelliğini kullanarak çözmenin (transform and conquer) en temiz örneklerinden biridir.

Kısa Tekrar Notları

- Complete binary tree: son seviye hariç tüm seviyeler doludur; son seviye soldan sağa dolar. Dizide ilk k göz doludur, boşluklar yalnız sondadır.
- Dizi gösterimi (1 tabanlı, 0. göz kullanılmaz):
  - Kök: `A[1]`
  - Sol çocuk: `A[2i]`, sağ çocuk: `A[2i+1]`
  - Ebeveyn: `A[i/2]` (tam sayı bölmesi, alta yuvarlama)
  - $0 \leq$ heap_size $\leq$ A.length
- Çocuğu olan düğümler dizinin ilk $\lfloor n/2 \rfloor$ gözündedir. Gerisi yapraktır.
- Boş küme de bir heap'tir.
- Karmaşıklıklar:

İşlem	Karmaşıklık
En büyüğü okuma (`return A[1]`)	$O(1)$
`MAX-HEAPIFY`	$O(\log n)$
`BUILD-MAX-HEAP`	$O(n)$
Extract max	$O(\log n)$
Increase key	$O(\log n)$
Insert	$O(\log n)$
Heap sort	$O(n \log n)$

Detaylı Açıklamalar

Heap mi, değil mi?

Derste üç ağaç incelendi (dizi gösterimiyle):

Dizi	Complete mi?	Heap koşulu?	Sonuç
10, 5, 7, 4, 2, 1	Evet	Evet	Heap
10, 5, 7, _, 2, 1	Hayır (son seviyede soldan boşluk var)	Bakmaya gerek yok	Heap değil
10, 5, 7, 6, 2, 1	Evet	Hayır (5 < 6)	Heap değil

- İlk örnek BST değildir: 7, 10'dan küçük olduğu hâlde sağdadır. Heap için bu sorun değildir.

Heap oluşturma: bottom-up

- Adım 0: Verilen elemanları sırayla diziye yaz. Bu bir complete binary tree verir.
- Adım 1: Son ebeveynden ($\lfloor n/2 \rfloor$) başlayarak köke doğru her düğüm için `MAX-HEAPIFY` çağır.
- `MAX-HEAPIFY`, düğümü çocuklarıyla karşılaştırır. En büyüğü yukarı alır. Aşağı inen değer için kendini özyinelemeli çağırır.

```
BUILD-MAX-HEAP(A, n)
  for i <- n/2 downto 1
    MAX-HEAPIFY(A, i, n)

MAX-HEAPIFY(A, i, n)
  l <- LEFT(i)
  r <- RIGHT(i)
  if l <= n and A[l] > A[i] then largest <- l else largest <- i
  if r <= n and A[r] > A[largest] then largest <- r
  if largest != i then
    exchange A[i] <-> A[largest]
    MAX-HEAPIFY(A, largest, n)
```

Örnek 1: `2, 9, 7, 6, 5, 8` ($n = 6$, ebeveynler 3, 2, 1)

Adım	İşlem	Dizi
Başlangıç		2, 9, 7, 6, 5, 8
i = 3	7 < 8, yer değiştir	2, 9, 8, 6, 5, 7
i = 2	9 > 6, 5; değişiklik yok	2, 9, 8, 6, 5, 7
i = 1	2 ↔ 9, sonra 2 ↔ 6	9, 6, 8, 2, 5, 7

Örnek 2 (kitap örneği): `4, 1, 3, 2, 16, 9, 10, 14, 8, 7` ($n = 10$, ebeveynler 5..1)

Adım	İşlem	Dizi
i = 5	16 > 7; değişiklik yok	4, 1, 3, 2, 16, 9, 10, 14, 8, 7
i = 4	2 ↔ 14	4, 1, 3, 14, 16, 9, 10, 2, 8, 7
i = 3	3 ↔ 10	4, 1, 10, 14, 16, 9, 3, 2, 8, 7
i = 2	1 ↔ 16, sonra 1 ↔ 7	4, 16, 10, 14, 7, 9, 3, 2, 8, 1
i = 1	4 ↔ 16, 4 ↔ 14, 4 ↔ 8	16, 14, 10, 8, 7, 9, 3, 2, 4, 1

- Min heap için aynı algoritma kullanılır. Yalnız karşılaştırmalar ters çevrilir (`largest` yerine `smallest`).

En büyük elemanı alma (extract max)

- Heap boşsa ($n < 1$) hata ver ya da dur.
- `max <- A[1]` ile kökü kopyala.
- Son elemanı köke koy: `A[1] <- A[n]`. Heap bilerek bozulur.
- Heap boyutunu bir azalt.
- `MAX-HEAPIFY(A, 1, n-1)` çağır. `max` değerini döndür.

Örnek: `20, 15, 8, 10, 5, 7, 6, 2, 9, 1` (slayttan)

Adım	Dizi (heap kısmı)
20 alınır, 1 köke gelir	1, 15, 8, 10, 5, 7, 6, 2, 9
1 ↔ 15, 1 ↔ 10, 1 ↔ 9	15, 10, 8, 9, 5, 7, 6, 2, 1
15 alınır, 1 köke gelir; 1 ↔ 10, 1 ↔ 9, 1 ↔ 2	10, 9, 8, 2, 5, 7, 6, 1

Bir düğümün değerini artırma (increase key)

- Yeni değer `key`, mevcut değerden büyük ya da eşit olmalıdır. Değilse hiçbir şey yapılmaz.
- `A[i] <- key` yapılır.
- `i > 1` ve `A[parent(i)] < A[i]` olduğu sürece düğüm ebeveyniyle yer değiştirir ve `i <- parent(i)` olur.
- Yukarı çıkarken aşağı inen eski ebeveyni tekrar kontrol etmeye gerek yoktur. O zaten eski alt ağacın en büyüğüydü.
- En kötü durumda yapraktan köke çıkılır: $O(\log n)$.

Örnek (kitaptan): `16, 14, 10, 8, 7, 9, 3, 2, 4, 1` dizisinde 4 değeri 15 yapılır.

Adım	Dizi
4 → 15	16, 14, 10, 8, 7, 9, 3, 2, 15, 1
15 ↔ 8	16, 14, 10, 15, 7, 9, 3, 2, 8, 1
15 ↔ 14	16, 15, 10, 14, 7, 9, 3, 2, 8, 1
15 < 16, dur	16, 15, 10, 14, 7, 9, 3, 2, 8, 1

- Toplam 3 karşılaştırma, 2 yer değiştirme.

Heap'e ekleme (insert)

- Heap boyutunu bir artır.
- Yeni son göze $-\infty$ yaz. Bu, heap koşulunu bozmaz.
- `HEAP-INCREASE-KEY(A, n+1, key)` ile değeri `key` yap. Değer gerekiyorsa yukarı çıkar.
- Karmaşıklık: $O(\log n)$.

Örnek (son heap üzerinde, tahtadan):

- 12 eklenir: 7'nin sağ çocuğu olur. 12 > 7, yer değiştirir. 12 < 15, durur.
- Ardından 20 eklenir: 9'un çocuğu olur. 9, 10 ve 16 ile sırayla yer değiştirip köke çıkar.

Top-down heap oluşturma

- Boş heap ile başla. Elemanları 1'den n'e kadar sırayla `insert` et.
- Aynı girdi farklı heap verebilir. Örnek girdi `1, 2, 3`:
  - Bottom-up: `3, 2, 1`
  - Top-down: `1` → `2, 1` → `3, 1, 2`
- İki sonuç da geçerli max heap'tir.

Heap sort

```
HEAPSORT(A, n)
  BUILD-MAX-HEAP(A, n)
  for i <- n downto 2
    exchange A[1] <-> A[i]
    n <- n - 1
    MAX-HEAPIFY(A, 1, n)
```

- Her adımda kökteki en büyük eleman, heap'in son gözüne taşınır. O eleman artık yerini bulmuştur ve heap dışında kalır.
- Karmaşıklık: $O(n) + (n-1) \cdot O(\log n) = O(n \log n)$.

Örnek: `16, 14, 10, 8, 7, 9, 3, 2, 4, 1` (heap hâlinde)

Heap boyutu	Heap kısmı	Sıralı kısım
10	16, 14, 10, 8, 7, 9, 3, 2, 4, 1	
9	14, 8, 10, 4, 7, 9, 3, 2, 1	16
8	10, 8, 9, 4, 7, 1, 3, 2	14, 16
7	9, 8, 3, 4, 7, 1, 2	10, 14, 16
6	8, 7, 3, 4, 2, 1	9, 10, 14, 16
5	7, 4, 3, 1, 2	8, 9, 10, 14, 16
4	4, 2, 3, 1	7, 8, 9, 10, 14, 16
3	3, 2, 1	4, 7, 8, 9, 10, 14, 16
2	2, 1	3, 4, 7, 8, 9, 10, 14, 16
1	1	2, 3, 4, 7, 8, 9, 10, 14, 16

- Sonuç: `1, 2, 3, 4, 7, 8, 9, 10, 14, 16`.
- İkinci örnek: `2, 9, 7, 6, 5, 8` önce `9, 6, 8, 2, 5, 7` heap'ine dönüşür. Ardından sırayla 9, 8, 7, 6, 5 sona taşınır. Sonuç: `2, 5, 6, 7, 8, 9`.

Üçlü heap (ternary heap)

- Her ebeveynin üç çocuğu olur. Örnek: kök 16, çocukları 14, 10, 8; 14'ün çocukları 7, 9, 3; 10'un çocukları 2, 4, 1.
- Yükseklik $\log_3 n$ olur. Ama her seviyede daha çok çocukla karşılaştırma gerekir. Bu yüzden genelde büyük kazanç sağlamaz. En çok ikili heap kullanılır.

Duyurular

- 2. ödev dersin ertesi günü tanımlanacak. Süre bir hafta, ayrıca 30 dakikalık geç teslim penceresi var.
- Vize iki hafta sonra, 9. haftada (bayram dönüşü), online.
- Fakültenin kararına göre final ve bütünleme sınavları yüz yüze olacak gibi görünüyor. Hoca bunun üst kurullarca değiştirilebileceğini, resmî duyuruların takip edilmesi gerektiğini söyledi.

Ders sonrası notlar

- **`MAX-HEAPIFY` slaytı:** Slaytta `if largest = i then exchange ...` yazıyor. Doğrusu `if largest ≠ i then` olmalıdır. Yer değiştirme yalnız en büyük eleman çocuklardan biriyse yapılır. Hoca da anlatırken "largest kendisi değilse yer değiştir" dedi.

- **`BUILD-MAX-HEAP` karmaşıklığı:** Hoca $O(n)$ sonucunu "$n/2$ kez fonksiyon çağrılıyor" diye açıkladı. Bu gerekçe eksik: her `MAX-HEAPIFY` $O(\log n)$ sürebilir, kaba hesap $O(n \log n)$ verir. Sonucun $O(n)$ olmasının nedeni, düğümlerin çoğunun alt seviyelerde olması ve az inmesidir. Sıkı analiz $\sum_h \frac{n}{2^{h+1}} O(h) = O(n)$ verir.
- **Heap yüksekliği:** Yükseklik $\lfloor \lg n \rfloor$'dir. 3 elemanlı ağacın yüksekliği 1'dir ($\lg 3 \approx 1.58$, aşağı yuvarlanır). Derste "yuvarlayınca 2" dendi; bu yanlış. 4 elemanda yükseklik 2'dir.
- **Extract max:** Hoca son elemanın "en küçük çocuk" olduğu için köke alındığını söyledi. Son eleman bir yapraktır ama en küçük olmak zorunda değildir. Köke alınmasının nedeni, diziden çıkarılabilecek tek göz olması ve complete yapının korunmasıdır.
- **Heap space:** Java'daki heap alanı, `new` ile oluşturulan nesnelerin tutulduğu dinamik bellek alanıdır. Değişkenlerin adreslerinin tutulduğu yer değildir. Veri yapısı olan heap ile ilgisi yoktur.
- **Heap sort:** Yerinde (in-place) çalışır ama kararlı (stable) değildir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.